

C16 — Specs, Engineering and Design Partnership

1. What interviewers test

"Walk me through how you'd take a feature from idea to launch." "Engineering says it will take three times longer than you expected. What do you do?" "Tell me about a disagreement with a designer." These questions look like behavior and process questions, and they are really tests of judgment about a PM's central job: turning a decision into shipped work with people who know things you do not. The interviewer wants to find out whether you can write down what a team should build and why, work with engineers and designers as partners, handle the friction that is normal in that partnership, and set and hold a quality bar.

Where it shows up. Inferred from the round structures candidates commonly report: execution and "working with others" questions at Google, Meta, Microsoft, Amazon, Apple, Netflix, Stripe and others, often as behavioral questions (C31) that probe how you worked with engineering, and as scenario questions ("the deadline is slipping") [unverified: candidate-reported]. Candidates for technical-leaning roles may get more depth on engineering partnership. On the job this is most of the week: specs, reviews, estimates, tradeoff conversations, launch checklists.

What is scored. Whether you are clear, collaborative and quality-minded Inferred. Interviewers listen for a spec that separates the problem from the solution and states what is out of scope; for estimates treated as ranges and negotiated through scope, not pressure; for respect for engineering and design expertise combined with a willingness to decide; for engagement with technical debt as an investment; for the other partners (legal, support, finance) involved early; and for a sensible quality bar and release decision.

Six observable behaviors:

1. You write or describe a spec that states the problem, goals, non-goals, requirements and metrics, and that someone could estimate and test.
2. You treat engineering estimates as ranges with assumptions, and negotiate scope rather than the estimate.
3. You bring engineers and designers into the problem early, and settle disagreements with evidence and clear decision rights.
4. You discuss technical debt with its cost and a plan, as an investment.
5. You involve legal, privacy, security, support and others at the right time.
6. You set a quality bar, triage defects by severity, and make a go or no-go call.

The failure this prevents. Six patterns end execution answers: a spec that is a list of features, a spec that is a 20-page document no one reads, pressuring engineers for shorter estimates, treating design as decoration, dismissing technical debt as an engineering complaint, and involving legal or security after the build.

2. The concept from first principles

C16.1 What execution is

Execution is turning a decision into a shipped outcome: the chosen problem and goal become a working change in users' hands, with the evidence to tell whether it worked. The PM's part is not to do the engineering or the design, but to make sure the team knows what outcome matters and why, the work is scoped and sequenced sensibly, obstacles are removed, decisions get made, quality holds, and everyone who needs to know does.

The split of ownership. The PM owns the *what and why*: the problem, the goal, the priorities, the success measure, and the decisions on scope. Engineering owns the *how* in the system: architecture, estimates, feasibility, and the quality of what is built. Design owns the *how* of the experience. Data science owns the rigor of measurement. Each has authority over their craft, and the PM has accountability for the outcome across them. This is why a PM's influence comes from clarity and trust more than from authority (C32).

When not to over-specify. For a small, reversible change in a trusted team, a few lines in a ticket and a conversation are enough. A heavy spec process for a button label is waste. Scale the documentation to the cost of being wrong (C02's decision classifier).

Counterexample. A PM writes a 20-page document with detailed screens and forty requirements, circulates it the night before kickoff, and treats it as final. Engineers find three requirements that conflict, designers find that the screens ignore an existing pattern, and nobody read the section on the goal. Two weeks in, the team is arguing about details and has lost sight of the problem. A PM who had written a two-page spec that opened with the problem, the goal and the non-goals, then met with the engineering lead and designer to shape the solution, would have had the same detail emerge as a shared product, not as a verdict.

Hidden risks. Two. *Confusing the document with the work*: the spec is a tool for alignment; a perfect spec that no one understands has failed. *Confusing authority with control*: telling people what to do produces compliance, not good work; the best specs leave room for the craft of the people who build.

C16.2 The PRD as craft: problem, goals, non-goals, requirements, metrics, risks

A **PRD** (product requirements document) is a written spec of the problem, goals, non-goals, requirements, metrics and risks that aligns a team on what to build. Teams call it other things (a spec, a one-pager, a brief, a press release and FAQ). Its job is alignment, and its length should match the stakes. FW-C16-01 is a skeleton.

Section	What it holds	A test
Problem	Who has what problem, with evidence; the cost of not solving it	Can a newcomer say it in one sentence, with no solution in it?
Goals and metrics	The outcome sought and how success will be measured, with a guardrail	Is there a number and a threshold (C05)?
Non-goals	What this project will <i>not</i> do	Would a reasonable person have expected it to be in scope?
Users and stories	Who is affected and the key scenarios	Are the stories testable (C16.3)?
Requirements	What must be true, in priority order (must, should, could)	Can an engineer estimate each and QA test it?
Design and technical notes	Constraints, links to designs, known approaches	Does it constrain the solution only where needed?
Risks, assumptions and open questions	What could go wrong; what is unproven; what is undecided	Is the riskiest assumption named, with a test (C15.6)?
Launch and measurement plan	How it will roll out and how results will be read	Who decides go or no-go, on what criteria (C17)?

A **non-goal** is something explicitly declared out of scope for a project. Non-goals are the most underused part of a spec and often the most useful: they prevent scope creep ([scope-creep]], C15), settle arguments in advance and tell people what they need not worry about. A good non-goal is one that a reasonable stakeholder might have assumed was included: "This release does not include editing from mobile web," not "We will not rebuild the company."

Bad versus good (FW-C16-02). A *bad* problem statement: "Users need a better search." It names a solution area and no problem. A *good* one: "Shoppers who search for a product by name get no results on 18% of queries, and 60% of them leave without buying (invented); we want to cut zero-result searches to under 8% without reducing relevance." It states who, what happens, the cost and the target. A bad requirement: "The system shall be fast." A good one: "Search results load in under 1.5 seconds at the 95th percentile on a mid-range phone on a typical mobile connection." It can be estimated and tested.

Length and form. A feature of a week may need a page. A multi-team launch may need a longer document with a short summary on top. A common alternative is to write the future announcement first ("working backwards," described publicly by Amazon [Public \[V: Amazon working-backwards material\]](#)), then the frequently asked questions that would follow, and derive requirements from them.

A living document. The PRD changes as the team learns. Keep a decision log (what was decided, by whom, why) so people do not reopen settled questions, and mark what changed after review.

Why it works. Writing forces clarity, and the written problem and non-goals prevent drift.

When not to write one at length. When the team already shares the context and the change is small and reversible.

C16.3 User stories, requirements and outcomes

A **user story** is a short description of a capability from the user's point of view, usually in the form "As a [user], I want [capability] so that [benefit]." It keeps the purpose attached to the request. **Acceptance criteria** are the specific, testable conditions a story must meet to be considered done. A common format is *Given* a starting situation, *When* an action occurs, *Then* an observable result follows.

Example.

- Story: As a shopper, I want to filter search results by size so that I only see items I can buy.
- Acceptance criteria: *Given* a results page with items in several sizes, *when* I choose size M, *then* only items available in M are shown and the count updates. *Given* no items in size M, *when* I choose it, *then* a message says so and offers to clear the filter. *Given* I have chosen size M, *when* I leave and return within a session, *then* my choice is kept.

A good story (FW-C16-03). The acronym INVEST sums the properties: *Independent* (can be built without another), *Negotiable* (the how is open), *Valuable* (to a user), *Estimable* (engineers can size it), *Small* (fits in a sprint), *Testable* (has acceptance criteria). When a story is too large to estimate, split it by workflow step, by user type, by data case or by simple-then-complex, never by technical layer alone (a "database story" delivers nothing a user can test).

Outputs and outcomes. A story describes an output (a filter). The goal describes an outcome (more shoppers find and buy an item they can use). Tie each story to the outcome, and measure the outcome. A team can ship every story and not move the goal; the PM's job is to notice and change course.

Why it works. Stories keep the user in view and acceptance criteria turn "done" into something checkable. **When not to use them.** For exploratory work or research, where the output is a learning and not a feature, state the question and the time-box instead.

C16.4 Working with engineering: estimates, pushback, tradeoffs, technical-debt conversations

Estimates are ranges. Engineers estimate the effort of work with uncertainty, and the uncertainty is largest early. A common pattern is that early estimates are off by a factor of two or more in either direction, narrowing as the work is understood (the "cone of uncertainty," first drawn by Barry Boehm in 1981 and named by Steve McConnell, who gives ranges of 0.25 to 4 times at the initial concept stage; Boehm presented the ranges as

judgment, not measured data) [V: Boehm, Software Engineering Economics, 1981; McConnell, Software Estimation, 2006 and 1998 naming; Bossavit, The Leprechauns of Software Engineering, on the origin as judgment]. Ask for a range and its assumptions ("5 to 12 weeks if the API exists; more if we need to build it"), the riskiest part, and what would narrow the range. FW-C16-04 sets out the conversation.

Use the outside view. If past similar projects took 1.1 to 2.0 times their estimates, with an average of 1.45 (invented), an 8-week estimate suggests planning for about 12 weeks ($8 \times 1.45 = 11.6$), within a range of 9 to 16 weeks, or cutting scope to meet 8. Do not pressure engineers to shrink the number: pressure produces optimistic estimates and hidden shortcuts. Negotiate *scope* (C15.4): "If we can only afford 8 weeks, what is the best version that fits?"

Handling pushback. When engineering says "that is not feasible" or "that is too much," ask what makes it hard and what would make it easier. Often there is a cheaper route to the same goal, or a hidden constraint you need to know. Treat pushback as information. Your part is to state the goal clearly so that engineers can propose alternatives, and to decide when a tradeoff is yours to make.

Technical debt. **Technical debt** is the accumulated cost of earlier shortcuts or aging design choices in a system, which makes later changes slower or riskier. It behaves like financial debt (FW-C16-05): the *principal* is the work to fix it, the *interest* is the extra cost paid every time the team works around it. Example (invented): a team of 4 engineers in 2-week sprints has 8 person-weeks per sprint. Workarounds for a fragile module cost 15% of capacity, 1.2 person-weeks per sprint (the interest). Fixing it costs 6 person-weeks (the principal).

Payback is $6 \div 1.2 = 5$ sprints, 10 weeks. Over six months (13 two-week sprints) the fix saves $13 \times 1.2 = 15.6$ person-weeks and costs 6: a net gain of 9.6. If the interest is only 8%, 0.64 person-weeks per sprint, payback takes 9.4 sprints and the net over six months is +2.3: still positive but marginal. The fix pays within six months as long as the interest is above about 5.8% of capacity ($6 \div 13 \div 8$). Treat the conversation as an investment with a return, compared with feature work using the same sizing (C15.5), not as a tax or a plea.

Learn enough to ask good questions. You do not need to code. You need the vocabulary and the shape of the system (C21): what the main components are, where data lives, what is hard to change, how releases work. Attend design reviews and ask "what is the riskiest part?" and "what would you cut if we had half the time?"

Why it works. Treating engineers as partners with ranges, assumptions and options yields better plans and earlier warnings. **When not to defer.** The PM still decides scope and priority; partnership does not mean avoiding decisions.

C16.5 Working with design and research: the triangle and how it fights

Product, design and engineering form a triangle, and its sides pull against each other in predictable ways. FW-C16-06 maps them.

Tension	What each side wants	A way through
Product and design	PM: ship the goal on time; Designer: get the experience right	Agree the quality bar by layer (C10.8); test the contested design with five users
Design and engineering	Designer: a custom, polished interaction; Engineer: standard components, lower cost	Compare user value, accessibility and maintenance; custom only where it matters
Product and engineering	PM: more scope; Engineer: fewer risks and time	Negotiate scope by the goal, with ranges (C15.4)

Roles. Designers explore options and own the quality of the experience; researchers produce evidence on users, ideally before the spec is final, not after; the PM frames the problem, brings evidence and criteria, and decides what ships. Bring designers into discovery, not only into drawing. Critique designs against the problem ("does this help a first-time user find checkout?"), not taste (C10.11).

When they fight. Disagreements are normal, and they are useful if they stay about the problem. First restate the shared goal. Then name the criteria each side is using (user value, cost, risk). Then seek evidence cheaply: a prototype test, a look at analytics, a technical spike. If the evidence does not settle it, the person with decision rights decides, explains, and the team commits ("disagree and commit," described publicly by Amazon Public [V: Amazon's Leadership Principles, Have Backbone; Disagree and Commit]), with a date to review if the decision proves wrong.

Why it works. Naming the tension and moving to evidence keeps the conflict productive.

When not to escalate. Most disagreements can be settled at the team level; escalate when the issue is outside the team's authority or blocks the date.

C16.6 Working with data science

Data scientists and analysts turn questions into evidence. Work with them early: before you write the metrics section, not after. Agree the *definition* of success (the primary metric, guardrails and counter metrics, C05), what needs to be instrumented ([instrumentation]), and how it will be read. For tests, agree the unit, sample size, duration and decision rule before launch (C14). Give them the decision, not only the data request: "We will ship if the lower bound of the lift is above 0.5 points and support contacts stay within 3%; can we measure that?" Ask for baselines, ranges and what the analysis cannot show. Avoid asking them to "find something in the data" that supports a decision already made; ask them to attack your hypothesis. Build a habit of shared dashboards and of pre-launch reviews of what will be logged.

C16.7 Other partners: legal, support, marketing, finance

Many projects depend on people outside the core team. Involving them at the right time avoids late blockers.

Partner	What they need from you	When to involve	Cost of late involvement
Legal and privacy	A description of data collected, claims made, terms changed	Before design is final; for anything touching personal data, payments, children or claims	A launch blocked or redesigned
Security	The architecture, data flows, third parties	At design review	High-risk findings found days before launch
Support and operations	What is changing, how users will ask, known issues, scripts	Before launch, with time to train	A surge of tickets with no answers
Marketing and communications	Positioning, dates, who the users are	When the launch plan forms	Messages that promise what the product does not do
Finance	Pricing, cost of the feature, forecast	When pricing or costs change	A price or cost you cannot sustain
Sales and customer success	What is coming, for whom, what to say	Before launch	Promises that the product cannot keep
Localization and accessibility	Strings, designs, requirements	During design (C09, C10)	Retrofitting

Why it works. Each partner holds a constraint that is cheap to meet early and expensive late. **When not to involve everyone.** Match the list to the change; a copy change needs few of them.

C16.8 Delivery rhythm: agile practices that matter

A **sprint** is a fixed, short period (commonly one to two weeks) in which a team commits to a set of work and delivers it in a usable state. It is a time-box ([[time-box]], C15). Agile practices are many, and a few matter most. FW-C16-07 lists them.

- **Short feedback loops.** Ship small, show working software often, and learn from it. This is the point of the method.
- **A visible backlog in priority order.** One list, ranked by the PM against the goal (C15), refined before each sprint so the next items are ready (clear, estimated, testable).
- **Planning.** The team chooses what fits its capacity and the sprint goal, not what a manager assigned.
- **Daily sync.** Short; to surface blockers, not to report to a manager.
- **A demo or review** of working software with stakeholders, to get feedback early.
- **A retrospective.** The team looks at how it works and changes one thing.
- **A definition of done.** The **definition of done** is the shared checklist a piece of work must meet before it counts as complete (for example, code reviewed, tests passing, accessibility checked, instrumented, documented). Without it, "done" means different things to different people and quality leaks out.
- **Limiting work in progress.** Fewer things started at once means more things finished.

The PM's role in the ceremonies is to bring the goal and the priority, answer questions, and make decisions quickly, not to run the meetings or to assign tasks. Drop ceremony that does not help: a long meeting that produces no decision is waste. Teams that ship continuously may prefer a flow-based method with a limit on work in progress and no sprints; the principles are the same.

Scrum and Kanban. Two named methods are worth recognizing by name. *Scrum* is a framework in which a small team of one product owner, one scrum master and developers works in sprints of a month or less, with four events (sprint planning, the daily scrum, the sprint review and the sprint retrospective) and three artifacts (the product backlog, the sprint backlog and the increment) [V: The Scrum Guide, Schwaber and Sutherland, November 2020]. The product owner is the Scrum role closest to a PM's, since it is accountable for the value of the product and for ordering the backlog Inferred [V: The Scrum Guide, 2020]. *Kanban* is a flow-based method: work moves across a visible board, and a limit on work in progress caps how much is started at once [V: Kanban Guide for Scrum Teams; university course notes on Scrum and Kanban]. In one line, Scrum fixes the rhythm and the roles, and Kanban fixes the limit on work in progress and lets work flow continuously. Teams often mix the two.

Why it works. Short loops catch wrong assumptions early and keep the team moving. **When not to follow the process.** When it substitutes for judgment; adapt to the team.

C16.9 Quality bar and working with QA

Quality is built in, not inspected at the end. Set the bar before work starts (C10.8), write it into the definition of done, and test continuously. QA engineers and testers bring a different mindset (what could go wrong?) and know the system's weak points; treat them as partners from the spec, so that acceptance criteria are testable.

Severity and the release decision. Classify defects by severity: *S1* (blocks a core task, loses data, creates a security or safety risk, or breaks accessibility for a group), *S2* (a significant problem with a workaround, or on a main path), *S3* (minor or cosmetic). A common release rule: no open *S1*; *S2* only with a known workaround and an owner; *S3* backlogged. Make the go or no-go call with the facts: count by severity, risk of each, time to fix, and the cost of delay. Worked example 5 does this with numbers.

Other release criteria. Accessibility basics (C10.6), performance on target devices (C09), instrumentation working, rollback tested (C17), and support trained.

Why it works. A stated bar and severity rules turn an argument into a decision. **When not to hold a release.** For *S3* issues and known low-risk defects; shipping with a known cosmetic flaw and a date to fix it beats delaying for perfection.

C16.10 Using AI tools in PM work: what counts as evidence

AI assistants can be used to draft specs, summarize research notes, generate test cases, review writing and analyze data. They change how fast a PM can produce text and analysis. They do not change what counts as evidence. FW-C16-08 sets the habits.

What they are good for. Drafting a first version from your bullet points; reformatting and editing; generating alternatives and counterarguments (a red team for your spec); listing edge cases and test cases; summarizing material you provide (interview notes, tickets) for you to check; explaining unfamiliar technical terms; writing queries or scripts that you then verify.

What they are not. A source of facts about your users or market. AI output is a draft or a hypothesis, not evidence: it may be wrong, may state invented numbers or sources with confidence, and may reflect biases in its training. Do not cite an AI answer as a source. If a number or claim matters, trace it to a primary source or to your own data.

Habits.

1. *Verify before you rely.* Check every fact, number, quote and citation against a source, especially anything that goes into a decision or a document others will rely on.
2. *Protect confidential data.* Know your company's policy; do not paste customer data, unreleased plans or personal information into tools that are not approved for it.
3. *Keep the decision human.* You own the judgment and the accountability; use the tool to widen the options, not to avoid choosing.
4. *Label AI-assisted work* where it matters (to colleagues, in documents) and never present synthesized text as user quotes.
5. *Watch for homogenized thinking.* A draft that sounds plausible can hide that it says nothing specific; add your own evidence and examples.
6. *Evidence from research stays from people.* An AI summary of interviews is a pointer to the transcripts; the transcripts and the participants are the evidence (C06). Synthetic "users" generated by a model are not users.

In interviews. If asked, say how you use AI tools: for speed on drafts and structure, with verification and confidentiality habits, and with clear ownership of the decision. Interviewers are testing judgment, not tool names (C24 and C25 cover building with AI).

Why it works. The habits capture the speed without the risk. **When not to use them.** When the data is confidential and the tool is not approved, or when the output cannot be verified at reasonable cost.

3. Frameworks

Eight frameworks, set out below. The decision memo (FW-C02-06), the success-metric template (FW-C05-06), the scope-negotiation script (FW-C15-07), the tradeoff triangle (FW-C15-05), the quality dial (FW-C10-05) and the experiment decision-rule table (FW-C14-05) are reused, not repeated.

FW-C16-01 · The PRD skeleton · Original

- **Inputs:** a decided problem and goal.

- **Outputs:** a one- to three-page spec with eight sections: problem, goals and metrics, non-goals, users and stories, requirements, design and technical notes, risks and open questions, launch and measurement.
- **Assumptions:** the team will read and discuss it; the scale of the document matches the stakes.
- **Tradeoffs:** writing costs time and saves rework; longer documents are read less.
- **Failure modes:** a feature list; no non-goals; a problem statement that contains the solution.
- **Worked example:** Worked example 1.

Sections: **Problem · Goals and metrics · Non-goals · Users and stories · Requirements (must/should/could) · Design and technical notes · Risks, assumptions, open questions · Launch and measurement.**

FW-C16-02 · The good-versus-bad spec test · Original

- **Inputs:** a draft spec.
- **Outputs:** answers to eight tests, a list of rewrites, and a verdict.
- **Assumptions:** an engineer and a tester can review it.
- **Tradeoffs:** a review takes an hour and saves days.
- **Failure modes:** reviewing format and not content; accepting vague requirements.
- **Worked example:** the rewrites in Worked example 1.

#	Test
1	Can a newcomer state the problem in a sentence, with no solution in it?
2	Is there a metric with a target and a guardrail?
3	Is there at least one non-goal a reasonable person might have expected in scope?
4	Can an engineer estimate each requirement?
5	Can a tester write a test for each requirement?
6	Are requirements prioritized?
7	Is the riskiest assumption named with a test?
8	Is it clear who decides what, and by when?

FW-C16-03 · The user story and acceptance criteria · Established

- **Origin:** user stories arose in extreme programming practice (Kent Beck and colleagues) and were popularized by Mike Cohn (*User Stories Applied*, 2004); the INVEST checklist is by Bill Wake (2003) [V: Beck with Andres, *Extreme Programming Explained*; Cohn 2004; Wake, *INVEST*, 2003].
- **Inputs:** a requirement from a user's point of view.

- **Outputs:** a story ("As a... I want... so that..."), acceptance criteria in given-when-then form, and an INVEST check.
- **Assumptions:** the team shares an understanding of the user and the goal.
- **Tradeoffs:** stories keep the purpose in view and can fragment a coherent experience; keep the outcome alongside.
- **Failure modes:** stories as tasks; no acceptance criteria; stories too large to estimate; solution written into the story.
- **Worked example:** Worked example 2.

INVEST: **Independent · Negotiable · Valuable · Estimable · Small · Testable.**

FW-C16-04 · The estimate conversation · Original

- **Inputs:** a piece of work and an engineering lead.
- **Outputs:** a range with assumptions and confidence, the riskiest part, what would narrow the range, an outside-view check, and options that trade scope for date.
- **Assumptions:** engineers can estimate with some uncertainty; history of similar projects exists.
- **Tradeoffs:** ranges are less comfortable than dates and more honest.
- **Failure modes:** pressuring for a smaller number; treating the low end as a commitment; no checkpoints.
- **Worked example:** Worked example 3.

Steps: **Ask for a range and assumptions → Ask for the riskiest part → Check against similar past projects → Offer scope options for the date you need → Set checkpoints to re-estimate.**

FW-C16-05 · The technical-debt conversation · Original

- **Inputs:** a piece of debt, the team's capacity and the cost of working around it.
- **Outputs:** the principal (fix cost), the interest (cost per sprint), the payback period and net over a year, a sensitivity on the interest, and a recommendation compared with feature work.
- **Assumptions:** the interest can be estimated from time lost, incidents or slowed delivery.
- **Tradeoffs:** paying down debt delays features; not paying slows all future work.
- **Failure modes:** treating debt as an engineering complaint; no numbers; fixing everything.
- **Worked example:** Worked example 4.

Formulas: **payback = principal ÷ interest per sprint; net over N sprints = N × interest – principal; break-even interest = principal ÷ (N × capacity).**

FW-C16-06 · The triangle conflict map · Original

- **Inputs:** a disagreement among product, design and engineering.

- **Outputs:** the tension named, each side's criteria, the cheapest evidence, and a decision owner and date.
- **Assumptions:** the disagreement is about the problem and not personalities.
- **Tradeoffs:** evidence costs time; unresolved conflict costs more.
- **Failure modes:** escalating first; deciding by rank; hiding the disagreement.
- **Worked example:** IQ-C16-04.

Steps: **Restate the shared goal → Name each side's criteria → Find the cheapest evidence → Decide (who, by when) → Commit and review.**

FW-C16-07 · The delivery rhythm · Original

- **Inputs:** a team and a goal.
- **Outputs:** a minimal set of practices (backlog in priority order, planning to capacity, short sync, demo, retrospective, definition of done, limit on work in progress) and the ones to drop.
- **Assumptions:** the team can inspect and adapt.
- **Tradeoffs:** ceremony can replace judgment; too little structure hides problems.
- **Failure modes:** copying a process; meetings with no decisions; no definition of done.
- **Worked example:** Worked example 5 shows the definition of done at release.

FW-C16-08 · The AI-assisted work check · Original

- **Inputs:** a task for which you would use an AI tool.
- **Outputs:** a decision to use or not, the habits applied (verify, protect data, own the decision, label, add evidence, keep research human) and a note of what remained unverified.
- **Assumptions:** you know your organization's data and tool policy.
- **Tradeoffs:** speed against risk of unverified output and leaked data.
- **Failure modes:** citing AI as a source; pasting confidential data; presenting synthesized text as user quotes.
- **Worked example:** IQ-C16-06.

Question	Habit
Will I rely on a fact or number?	Verify against a primary source
Is the data confidential?	Use only approved tools; otherwise do not paste it
Whose judgment is this?	Mine; the tool widens options
Does it need labeling?	Label AI-assisted work where it matters
Is it user evidence?	Only from people and transcripts

4. Answer architecture

The execution answer, for a 20-minute round

Step	Minutes	What to produce
Clarify scope and team	2	The goal, the team, the date, what is fixed
The spec	5	Problem, goals and metrics, non-goals, requirements in priority order, risks
Partners and roles	4	Engineering, design, data science and the other partners, and when each is involved
Estimates and tradeoffs	4	Ranges, assumptions, the outside view, scope options; technical debt if relevant
Rhythm and quality bar	3	Delivery practices that matter; definition of done; severity rules
Launch readiness and measurement	2	Go or no-go criteria; how results will be read

Pacing rules. If the interviewer asks about a specific problem (a slipping date, a conflict), spend the time on that and use the structure briefly. If you run behind, keep the spec outline and the way you handle the main friction.

How to say it. Be concrete: "I'd write a two-page spec with the problem, a metric with a threshold, three non-goals and the requirements in priority order. I'd review it with the engineering lead and designer before anyone commits."

How to close. Two sentences: "I'd align the team on a short spec with explicit non-goals, treat estimates as ranges and negotiate scope, bring design, data science and the other partners in early, and hold a stated quality bar with a clear go or no-go rule. I'd change course if [signal]."

Type: walk me through feature to launch

Use the full structure; spend the most on the spec and the partners.

Type: engineering pushback or a slipping date

Use the estimate conversation (FW-C16-04) and the tradeoff triangle (C15): ask for the range and the riskiest part, check against the outside view, offer scope options, set checkpoints.

Type: design and engineering disagree

Use the conflict map (FW-C16-06): shared goal, criteria, cheapest evidence, decision owner.

Type: technical debt or a refactoring request

Use FW-C16-05: principal, interest, payback, sensitivity, and compare with features.

Type: a release decision

State the quality bar, triage by severity, count the facts and make the call with a rollback plan.

Type: using AI tools

Say what you use them for, how you verify, how you protect data, and that the decision stays yours.

Common interviewer interventions, and what to do

Interviewer says	What it means	What to do
"What's in your spec?"	They want the content	Walk the eight sections, with non-goals
"What won't you build?"	Non-goals	Give two non-goals a stakeholder might have expected
"Engineering says it's six weeks."	An estimate	Ask for the range, the riskiest part and scope options
"Can't you just add people?"	Brooks's law	Show the onboarding cost; offer scope instead
"The designer disagrees."	Conflict	Restate the goal, name the criteria, find cheap evidence, decide
"There are 40 open bugs. Ship?"	Quality bar	Triage by severity; apply the release rule; count the risk
"Did AI write this?"	Judgment on tools	Say how you use it, verify, and own the decision

5. Worked examples

All five examples are *Illustrative*: the products are fictional or use a product type as the setting only, and every figure is an invented assumption. The arithmetic was computed. Each is written as notes you could say aloud.

Worked example 1 — A spec for admin audit logs in an enterprise product (Illustrative)

Setting: an enterprise collaboration product of the kind Microsoft offers. No fact about any real product is claimed.

The request. Large customers ask for "audit logs." Security teams at two of them have said they need them for compliance reviews.

The problem statement, bad and good (FW-C16-02). *Bad*: "Customers need audit logs." It names a solution. *Good*: "IT administrators at organizations with compliance reviews cannot answer 'who changed this permission, and when' without opening a support ticket."

Today a review requires an average of 6 support tickets per quarter per affected customer and delays audits by about 2 weeks (invented). We want administrators to answer these questions themselves, in under 5 minutes."

Goals and metrics (FW-C05-06). Primary: the share of eligible administrators who view the log at least once within 60 days of release (target 35%, invented). Input: median time to find a given event (target under 5 minutes). Guardrails: page performance (95th percentile load time must not rise by more than 2%), storage cost per customer within budget. Counter metric: support tickets about permission changes (should fall; if not, the log is not answering the question).

Non-goals. (1) Real-time alerts on events (a later release). (2) Customer-defined retention periods in version 1 (fixed at 90 days). (3) Export to third-party security tools in version 1. A reasonable stakeholder might have expected each of these, so naming them prevents arguments.

Requirements, in priority order. *Must:* record sign-ins, permission changes and file shares with who, what and when; search by person and date range; keep 90 days; show events within 5 minutes of occurrence. *Should:* export to a spreadsheet file. *Could:* saved filters.

Risks, assumptions, open questions. The riskiest assumption is that administrators will look at the log (a log nobody reads is cost without value). Test: interviews with ten administrators and a prototype shown to five, before the build. Other assumptions: the existing event stream carries the data needed (a technical spike will check). Open questions: retention rules by country (legal), storage cost (finance), how deleted users appear (design).

Partners. Legal and privacy for retention and personal data; security for access to the log itself (who may read it); support for training; engineering for the event stream; design for search and filters; data science for the instrumentation of log views.

Launch and measurement. Staged: ten design-partner customers, then general availability; go or no-go on the criteria in the spec (no S1 defects, search performance within target, support trained).

What this shows. The spec fits on two pages, separates the problem from the solution, names the non-goals and the riskiest assumption, and could be estimated and tested.

Worked example 2 — User stories and acceptance criteria for "share a list" (Illustrative)

A weak story (invented). "As a user, I want a share button." It states a control, not a purpose, so it cannot be tested beyond whether a button exists.

A better story (FW-C16-03). "As a shopper planning a trip with a friend, I want to share my packing list so that my friend can see it and tick things off." It names who, what and why.

Too large. "Manage who can see or edit my list" is too big to estimate (it hides several behaviors). Split by workflow step into three stories, each valuable and testable on its own:

1. *View-only link.* Given I have a list, when I choose "share" and then "view only," then a link is created that lets anyone with it see the list but not change it.

2. *Edit access for a named person.* Given I have shared a list, when I invite a person by email with edit access, then they can add and tick items and I see their changes within 10 seconds.
3. *Revoke access.* Given a person has access, when I remove them, then they can no longer open the list and see a message that access ended.

INVEST check. Each story is independent (1 can ship without 2), negotiable (the how is open), valuable (a user sees a benefit), estimable (engineers can size each), small (a few days) and testable (the criteria are observable). The original "manage who can see or edit" failed small and estimable.

Outcome, not only output. The stories deliver outputs. The outcome is more shared lists that two people use (for example, the share of shared lists with edits by both people within a week). The team should read that number after launch, and if the stories ship without it moving, ask why.

What this shows. Splitting by workflow step and writing acceptance criteria turned a vague request into three testable increments.

Worked example 3 — The estimate conversation: "that's eight weeks" (Illustrative)

The situation. The PM hopes to launch in 5 weeks. The engineering lead estimates 8 weeks for 10 stories (about 0.8 weeks each).

Step 1: ask for a range and assumptions (FW-C16-04). The lead says 5 to 12 weeks: 5 if an existing service can be reused, 12 if a new data model is needed, 8 as a middle view. The riskiest part is the data model. What would narrow it: a two-day technical spike.

Step 2: the outside view. The team's last six similar projects took 1.1, 1.2, 1.3, 1.5, 1.6 and 2.0 times their estimates (invented): a median of 1.4 and a mean of 1.45. Applied to 8 weeks, that is 11.2 to 11.6 weeks. The honest planning range for the full scope is about 11 to 13 weeks, not 5.

Step 3: do not negotiate the estimate; negotiate the scope. Options for a date:

Option	Scope	Estimate (8-week basis)	With the outside view
A. Full scope	10 stories	8 weeks	about 11.6 weeks
B. Core scope	6 stories	4.8 weeks	about 7.0 weeks
C. Smallest useful slice	4 stories	3.2 weeks	about 4.6 weeks

To hit 5 weeks with confidence, the answer is option C: four stories, which deliver the core of the user's main job, and defer the rest to a second release. Option B fits 7 weeks.

Step 4: set checkpoints. Do the two-day spike first. At the end of week 2, re-estimate with the new information. If the spike shows the existing service can be reused, the range moves toward 5 and more scope fits; if the data model is needed, the plan is already the smaller slice.

What I would say. "I don't want to push the estimate down; I'd rather make the date true. With our history, the full scope is likely closer to 11 weeks. For 5 weeks, I'd ship the four-story core and add the rest in a follow-up. Let's run the spike this week and check again on Friday."

What this shows. Pressure on the number would have produced a hidden overrun. The range, the outside view and the scope options produced a plan that could be met.

Worked example 4 — The technical-debt conversation (Illustrative)

The situation. An engineering lead asks for a full sprint to rework a fragile billing module. The PM must weigh it against features.

The numbers (FW-C16-05) (invented). The team has 4 engineers in 2-week sprints: 8 person-weeks per sprint. The lead estimates that workarounds, bug fixes and slow changes around the module cost about 15% of capacity, 1.2 person-weeks per sprint (the interest). The rework costs 6 person-weeks (the principal). Payback: $6 \div 1.2 = 5$ sprints, or 10 weeks. Over six months (13 two-week sprints), the rework saves $13 \times 1.2 = 15.6$ person-weeks and costs 6, a net gain of 9.6 person-weeks, which is more than a sprint's worth of feature work.

Sensitivity. The interest is the uncertain number. If it is only 8% of capacity (0.64 person-weeks per sprint), payback is 9.4 sprints and the net over six months is $13 \times 0.64 - 6 = +2.3$. The rework pays within six months as long as the interest exceeds about 5.8% of capacity ($6 \div 13 \div 8$). Ask the team to measure it: how many tickets, incidents and hours trace to the module over the last quarter?

Beyond capacity. Debt also carries risk: this module handles billing, so an outage or error costs money and trust. That belongs in the case alongside the time cost (a defect rate and its cost, for example).

Compare it with features (C15.5). If the best alternative use of 6 person-weeks returns, say, \$90,000 in expected value, and the rework frees 15.6 person-weeks over six months (which could return about \$234,000 at the same rate), the rework is worth more.

The decision. Do the rework, but not as a full sprint of nothing else: split it into two stages of three person-weeks each, with a measure of the interest after the first stage (hours lost on the module). If the measured interest is under the break-even, stop after stage one. Tell the sales and support teams what the sprint will and will not deliver, and the date features resume.

What this shows. The conversation became an investment case with a payback, a sensitivity and a way to check it, instead of a request that sounded like a complaint.

Worked example 5 — A release decision: 40 open bugs (Illustrative)

The situation. Eight working days before launch, QA reports 40 open defects (invented): 3 S1, 9 S2, 28 S3. Two engineers are available for fixes: 16 person-days.

The release rule, set in advance (C16.9). No open S1. S2 allowed only with a documented workaround and an owner. S3 backlogged.

Triage. Of the 9 S2 defects, 5 are on the core path (checkout) and 4 are in secondary screens with workarounds. Estimates: each S1 takes about 2 days, each core-path S2 about 1.5 days.

The arithmetic. S1: $3 \times 2 = 6$ person-days. Core-path S2: $5 \times 1.5 = 7.5$. Total 13.5 of 16 available, leaving 2.5 person-days (16%) as a buffer. The 4 secondary S2 defects ship with workarounds, and the 28 S3 defects go to the backlog.

Sensitivity. If the S1 fixes take 4 days each instead of 2, the total is $3 \times 4 + 7.5 = 19.5$, which exceeds 16. Then something must give: delay by about two days, or fix only the S1s and the three most severe core-path S2s, or launch to a smaller group first. Re-estimate after the first S1 is fixed (day 2): if the first takes 4 days, the rest are likely similar, and the decision moves up.

The decision and the safety net. "Fix the three S1s and the five core-path S2s, ship with the four secondary S2s and 28 S3s documented, and launch in stages: 10% of users for two days with guardrails on checkout completion and errors, then 50%, then all. Rollback is tested before day one. Support has the workarounds. Go or no-go meeting on day 7 with the severity counts and the staged results." Accessibility and security checks are release criteria, not defects to trade.

What I'd tell the stakeholders. "We will ship on the date with no S1 and the core path clean. Four secondary issues have workarounds and are scheduled for the first patch. If the first S1 fix takes longer than expected, I will tell you by day 2, and the choice will be a two-day delay or a smaller first release."

What this shows. A rule set in advance, severity triage, capacity arithmetic and a staged launch turned a long bug list into a decision.

6. Baseline, Strong, Standout

Prompt: "Outline the spec for a 'saved filters' feature on a search page, and say what you would not include."

Baseline answer

"I'd write down the features: a button to save a filter, a list of saved filters, and a way to apply or delete them. I'd also add notifications when new results match and sharing of filters with others."

What is wrong: a feature list with no problem, user or metric. The "not include" part is missing (everything that came to mind went in). It cannot be estimated or tested.

Strong answer

"I'd start with the problem: people who run the same search repeatedly, such as a buyer checking daily for new listings, have to set the same filters every time. I'd state the goal with a metric: for returning searchers, cut the time to reach their results from about 40 seconds to under 10, and raise the share who return within a week. Guardrail: page performance. Non-goals: notifications about new matches (a later release), sharing filters with other people, and syncing across accounts.

"Requirements in priority order: must save the current filters with a name, list and apply saved filters in one tap, rename and delete; should keep the saved filters across devices; could let people reorder them. Each requirement has acceptance criteria that QA can test. I'd name the riskiest assumption (that people repeat searches enough to use saved filters) and check it with search logs before building.

"I'd review the spec with the engineering lead and designer before committing, and ask for an estimate as a range."

Standout answer

Everything in the Strong answer, plus:

"I'd decide how large the spec needs to be: for a small feature, two pages with the problem, metrics, non-goals and stories. I'd put the decision log at the top so that settled questions stay settled.

"On non-goals, I'd choose ones people would plausibly expect: notifications are the likely next request, and saying so now avoids scope creep. I'd also say that the first version stores filters on the device only, if that halves the work, and that cross-device sync follows if usage justifies it, which makes the minimum viable version the test of the riskiest assumption.

"I'd involve privacy early, because saved searches can reveal what a person is looking for, and they may be stored; I'd confirm where they live and how long. I'd ask data science to define how we will tell that the feature changed behavior, not just that people used it: the share of saved-filter users who return and the time to results compared with similar users without it, with a holdout if possible.

"The quality bar: accessible controls, an empty state that explains what saved filters are, and an error state if saving fails. Release criteria: no S1 defects, search performance within 2% of today, rollback tested.

"What would change my mind: if search logs show that fewer than 5% of users repeat a search, I'd drop the feature or fold it into a lighter 'recent searches' list."

What moves an answer up a band

From Baseline to Strong	From Strong to Standout
States the problem, a metric and non-goals	Chooses non-goals by what stakeholders would expect and sizes the first version to test the riskiest assumption

From Baseline to Strong	From Strong to Standout
Writes testable requirements in priority order	Involves privacy, data science and quality criteria early
Reviews with engineering and design before committing	Names what would make you drop or shrink the feature

7. Company lens

All claims describe what companies say publicly or what candidates commonly report, not insider knowledge. Interview formats change [as of 2026-10].

Company lens: Microsoft [as of 2026-10]. Microsoft's products serve enterprise customers whose administrators, security teams and compliance officers are partners in what ships Inferred [unverified: author's reasoning from public product descriptions; no source checked]; the company publishes inclusive-design material Public. Practical consequence: in a spec, name the administrator and the compliance requirements as users, involve security and legal early, and say how you would stage a rollout to design-partner customers.

Company lens: Google [as of 2026-10]. Google has described a culture of design documents reviewed by engineers before major work, and has published its approach to reliability and postmortems Public [V: Google SRE book on postmortems; design documents not checked]. Candidates commonly report execution questions probing how you work with engineers at scale Inferred. Practical consequence: expect to be asked how you would invite technical review, how you treat estimates, and how you decide go or no-go.

Company lens: Amazon [as of 2026-10]. Amazon has described writing a future press release and FAQ before building, narrative documents in place of slides, and a principle of disagree and commit Public [V: Amazon working-backwards material; 2017 shareholder letter; Leadership Principles]. Practical consequence: be ready to write the customer's announcement and the hardest questions it would draw, and to describe a time you disagreed and committed.

Company lens: Meta [as of 2026-10]. Meta has publicly changed its engineering motto over time, from moving fast and breaking things to moving fast with stable infrastructure, which shows the tension between speed and quality Public [V: Fortune, 16 February 2022, on Meta's values and the 2014 motto change]. Candidates commonly report questions about balancing launch speed with quality and about working with engineers on tight timelines Inferred. Practical consequence: state your quality bar and your staged-launch practice.

Company lens: Apple [as of 2026-10]. Apple integrates hardware, software and services, and third-party guides report that execution prompts concern cross-functional work in which design and engineering craft are given great weight Inferred [unverified: candidate-reported]. Practical consequence: show respect for craft, describe how you protect quality when schedules press, and name how you handle dependencies across hardware and software.

Company lens: Netflix [as of 2026-10]. Netflix's published culture material describes high-trust, high-accountability teams in which informed individuals decide Public [V: Netflix culture memo, June 2024]. Practical consequence: describe who decides, on what context, and how you give engineers context instead of instructions.

Company lens: Stripe [as of 2026-10]. Stripe is widely described as having a strong writing culture and as emphasizing API design and documentation quality Inferred [V: The Pragmatic Engineer, Inside Stripe's Engineering Culture Part 2 (strong culture of writing); Slab interview with Stripe's former Documentation Manager]. Practical consequence: write specs that engineers and developers could use as documentation, with explicit contracts, error behavior and non-goals.

8. Common mistakes

#	Mistake	How to detect it	Fix
1	A spec that is a feature list	No problem, metric or non-goals	Use the PRD skeleton; start with the problem
2	A problem statement that contains the solution	"Users need a better X"	Rewrite with who, what happens and the cost
3	No non-goals	Scope arguments keep returning	Name two things a stakeholder might expect
4	Untestable requirements	"Fast," "easy," "intuitive"	State a measurable threshold and a condition

#	Mistake	How to detect it	Fix
5	Pressing for a smaller estimate	The number shrinks and the date slips	Ask for a range; negotiate scope
6	Adding people late	"We'll add engineers"	Do the onboarding arithmetic; cut scope
7	Treating technical debt as a complaint	"Not now" with no numbers	Principal, interest, payback; compare with features
8	Design as decoration	Designers brought in after the spec	Include design in discovery; critique against the problem
9	Late partners	Legal or security hears two weeks before launch	Use the partner table; involve them at design
10	No release rule	The go or no-go is argued bug by bug	Set severity rules in advance; stage the launch

9. Mental models applied

Four models, each tied to a decision in this chapter. The fence idea is from C08.

Conway's law (MM-conway). Organizations tend to produce systems that mirror their communication structure [V: Conway, 1968, as quoted in several sources]. If three teams must agree to ship a feature, the feature will have three seams. Use it to shape the work: put the people who must make decisions together, and expect the architecture to follow the team boundaries, for better or worse.

Hofstadter's law (MM-hofstadter). It always takes longer than you expect, even when you take into account Hofstadter's law. The point is practical: add margin, use the outside view (C15), and re-estimate at checkpoints, since confident early estimates are the least reliable.

The fence, applied to code (MM-chestertons-fence, C08). Before removing or rewriting something that looks like clutter, learn why it exists. A strange rule in a billing module may encode a legal requirement or a fix for an old failure. The technical-debt conversation should start with understanding, not contempt.

Make the change easy, then make the easy change (MM-easy-change). A phrase attributed to Kent Beck [V: Kent Beck, Software Design: Tidy First? newsletter, 30 January 2025, repeats the line; also quoted in Paranj, Test Driven Development in Ruby, 2017; date of first appearance not checked]: when a feature is hard to add, first restructure so that it becomes easy, then add it. For a PM, it justifies sequencing a small refactoring before a feature when the feature would otherwise be built on a weak base.

Engineering, design and data-science perspectives

- **Engineering.** Engineers want a clear goal, early involvement, stable priorities and respect for estimates. They can tell you which parts are risky and where the cheaper

route is. Bring them into problem framing and listen for "that's hard because...", which is often the start of a better design.

- **Design and research.** Designers want to understand the problem and the user, not just receive a list. Research wants to feed the spec, not audit it. Share the goal and constraints early, and ask for options.
- **Data science.** Data scientists want clarity on the decision, the definition of success and what will be logged. Involve them before the metrics section is final, and again before launch to check instrumentation.
- **Legal, security, support and operations.** They hold constraints and the cost of late surprises. Give them a short brief with what is changing and when, and ask what they need to see by when.

Checklist: before you start building

1. Can the problem be stated in one sentence with no solution in it?
2. Is there a metric with a target, a guardrail and a counter metric?
3. Are there non-goals a reasonable person might have expected in scope?
4. Is each requirement estimable and testable, in priority order?
5. Did engineering give a range, with assumptions, and did I check it against history?
6. Is the riskiest assumption named with a cheap test?
7. Have legal, security, support, design and data science seen the plan at the right time?
8. Are the quality bar, severity rules and go or no-go owner set, with a rollback plan?

10. Case studies

CS-C16-01 · The spec that said "improve search" · Illustrative · Beginner

Fictional composite. Every number is invented (invented).

Situation. An e-commerce team of four engineers (24 person-weeks over three sprints) is told by a stakeholder, in one line of a ticket, to "improve search." The PM adds nothing to the ticket. Engineers read it as a quality project and build type-ahead suggestions.

Constraints. A fixed team, a quarter-end review, and no written problem statement or metric.

Data. After the team ships type-ahead at the end of sprint three, the search metrics that the stakeholder cares about barely move: 18% of queries still return no results, and 60% of the people who see no results leave without buying. Interviews with the stakeholder reveal the real goal: cut zero-result queries, which were losing sales. Of the 24 person-weeks, 9 (37.5%) went into type-ahead, which did not address the goal.

Problem definition. As an observable problem: "The team built what the words suggested, not what the stakeholder needed, because the ticket stated neither the problem nor the goal."

Analysis. The spec failed the good-versus-bad test (FW-C16-02) on every count: no problem sentence, no metric, no non-goals, no testable requirements. Engineers filled the gap with a reasonable interpretation, and nobody could tell it was wrong until after the build. The cost of a clarifying conversation at the start, about two hours with the stakeholder, was far smaller than 9 person-weeks of rework. Once the goal was stated, the solution turned out to be small: handle common misspellings and synonyms so that fewer queries return nothing, about 4 person-weeks.

Options.

Option	Optimizes	Costs
A. Keep the ticket and iterate on what shipped	No rework conversation	The goal stays unmet
B. Write a two-page spec with the problem, a metric, non-goals and requirements, then rebuild the scope	The actual goal	A conversation; discarded work
C. Ask the stakeholder to specify the solution	Speed	A solution chosen by someone not close to the system

Tradeoffs. B costs a short meeting and gives the team something to estimate and test. C hands the design to the wrong person. A repeats the problem.

Decision. Choose B. The spec: *Problem*: 18% of searches return no results and 60% of those users leave. *Goal*: cut zero-result queries from 18% to under 8%, without lowering relevance (guardrail: click-through on results). *Non-goals*: no new ranking model, no type-ahead changes this quarter. *Requirements*: spelling correction for the top misspellings, a synonym list for the top 200 terms, and a "no results" page that suggests related searches. *Primary metric*: the zero-result rate.

Outcome. In this fictional scenario the zero-result rate fell from 18% to 7% (invented), delivered in 4 person-weeks, and click-through on results did not change. The type-ahead work stayed shipped as a modest improvement. The PM added a two-page spec template to the team's process.

Alternative outcomes. If the zero-result rate had fallen only to 12%, the team would have examined the remaining queries (rare products, long queries) and set a second goal. If the stakeholder had wanted something else again, the spec would have exposed the disagreement in the first meeting and not after a quarter. The lesson: the cheapest place to fix a project is the first page of the spec.

CS-C16-02 · HealthCare.gov, 2013: launching without stable requirements or verified performance · Verified · Advanced

Sources (types): the US Government Accountability Office report GAO-14-694, "Healthcare.gov: Ineffective Planning and Oversight Practices Underscore the Need for Improved Contract Management" (31 July 2014), read through GAO's highlights and page

extracts, and the GAO testimony of 31 July 2014 (GAO-14-824T), read in full (primary); GAO's September 2014 report on security and privacy controls (GAO-14-730), read through its highlights; and news coverage of the report (CNBC, FEDweek, KFF Health News, Fierce Healthcare). The analysis is the author's. This case concerns a government program built by contractors; it is used here for what it shows about specs, decision rights and release criteria, and it makes no comment on policy. Re-read in October 2026.

Situation. The Centers for Medicare & Medicaid Services (CMS) contracted in September 2011 for the systems behind the federal health insurance marketplace, accessed through Healthcare.gov. Enrollment opened on 1 October 2013, a date set by the program's legal timeline. When it opened, many users experienced website failures, errors and slow responses. GAO's review found that CMS had developed Healthcare.gov and its related systems without effective planning or oversight practices, and that costs rose and schedules slipped, primarily because requirements kept changing, a problem made worse by inconsistent oversight.

Constraints. The program was a first-of-its-kind federal marketplace, a complex effort under a fixed launch date. The work was divided among multiple contractors, under a cost-plus-fixed-fee arrangement that GAO described as high risk for the government, while saying it may have been a reasonable choice in the circumstances [V: GAO-14-824T]. The date could not move easily: it was tied to the enrollment period.

Data. As reported by GAO: CMS issued task orders for the federally facilitated marketplace (FFM) and the data hub when key technical requirements were still unknown, including the number and composition of states to be supported and the number of potential enrollees, and proceeded with the award process before requirements were stable; CMS officials said that meeting deadlines drove several planning decisions, including the choice of a cost-plus-fixed-fee contract; CMS did not develop a required acquisition strategy; from September 2011 to February 2014 estimated FFM development costs rose from an initial obligation of \$56 million to more than \$209 million, and data hub costs from \$30 million to almost \$85 million, due primarily to changing requirements exacerbated by inconsistent oversight; CMS moved an assessment of FFM readiness from March to September 2013, weeks before launch, and did not receive required governance approvals, so it launched "without verification that it met performance requirements"; because of unclear roles, GAO found about 40 instances in which program staff inappropriately authorized contractors to spend funds, totaling over \$30 million; CMS gave the FFM contractor written concerns in April and November 2013, moved program staff to the contractor's offices in September 2013, and ultimately declined to pay about \$267,000 in requested fee; and in January 2014 CMS awarded Accenture Federal Services a contract estimated at \$91 million to continue FFM development, whose costs had risen to over \$175 million by 5 June 2014 with key capabilities, including the financial management module, still unavailable [V: GAO-14-824T; GAO-14-694 highlights]. CMS reported obligating \$840 million for the development of Healthcare.gov and its supporting systems as of March 2014 [V: GAO-14-824T].

Problem definition. *The author's reading.* The project had a fixed date, unstable requirements, divided accountability and a late and reduced test of whether the system worked. As an observable problem: "The system was launched to the public on a fixed date without verification that it met its performance requirements."

Analysis. *The author's analysis.* Read through this chapter's tools, several of its themes appear at scale. *Specs:* work was ordered when key requirements were unknown and requirements kept changing; a stable statement of the problem, the goals and the non-goals (FW-C16-01) is the antidote to churn, and the report links the cost and schedule growth to changing requirements. *The triangle:* with the date fixed (by law), scope and quality were the levers; the evidence suggests quality verification was what gave (FW-C15-05). *Decision rights:* confusion about who could decide is a failure of the roles and decision ownership that C16.5 and C32 describe. *Quality bar and release criteria:* a go or no-go rule with stated performance criteria, and a check that they were met, would have exposed the gap; the readiness review that was moved later is the very mechanism that should have caught it (C16.9). *Launch approach:* a full public launch left no room to learn at small scale; a staged rollout (C17) can limit exposure when a date cannot move, though it may not have been possible within the legal constraints [inference]. *Partners:* security and privacy were later the subject of a separate GAO report (GAO-14-730, September 2014), which found that Healthcare.gov had weaknesses when first deployed, including incomplete security plans and privacy documentation, incomplete security tests and no alternate processing site [V: GAO-14-730 highlights], a reminder that partners and verification belong in the plan early.

The case does not show that any individual made a wrong choice, and it does not show what a different approach would have achieved. It shows what the public record says about the conditions: a hard date, unstable scope, split accountability, and a verification step that was late and incomplete.

Options. *The author's reconstruction of choices a program team could have faced; not a record of CMS's deliberations.*

Option	Optimizes	Costs
A. Launch on the date with the full scope, as happened	Meeting the date	Unverified performance; failures at the public launch
B. Delay the launch until performance is verified	Quality	Missing a legally set date; consequences for the program
C. Launch on the date with a reduced scope or limited exposure, with verified performance on the core path	Meeting the date and protecting the core	Fewer features or users at first; political and legal constraints may have limited this
D. Freeze requirements early and test integrated performance well before launch	Stability and evidence	Earlier decisions; an earlier hard conversation about scope

Tradeoffs. A met the date and carried the quality risk. B protected quality and broke the date. C is the middle path and the one the triangle favors when the date is fixed; D is the cheapest in the long run and the hardest to do politically, because it forces scope decisions early.

Decision. As documented, the system launched on 1 October 2013 without verification that it met performance requirements.

Outcome. Reported: many users could not complete enrollment at launch; costs and schedule grew; GAO recommended better contract management and governance. How the system later improved, and the effect of recovery efforts, are outside the sources used here.

Alternative outcomes. *Inference.* A frozen requirements baseline and early integrated testing would have surfaced the performance problems when there was time to fix them. A limited launch might have reduced the number of people affected by early failures. A reader should be careful with hindsight: the same facts would be read differently if the launch had worked. The method to carry away: write the problem and non-goals so that requirements stop moving, set the release criteria in advance and verify them, make decision rights explicit, bring partners in early, and when the date is fixed, reduce scope or exposure before you reduce verification.

11. Interview questions and model answers

Seven questions: two Beginner, two Intermediate, three Hard. Model answers are **Illustrative** (composite candidates), every figure is an assumption, and the arithmetic was computed. Each is written against the six behaviors in section 1. Where a real company's product is the setting, the answer reasons from public information and states no internal fact. The answers are fuller than a first pass in a real round.

IQ-C16-01 · Beginner · execution · EX

Prompt. What goes into a good PRD?

Rubric.

- **Baseline:** "The requirements and the designs."
- **Strong:** Lists the sections (problem, goals, non-goals, requirements, metrics, risks) and says why each exists.
- **Standout:** Distinguishes good from bad examples, scales length to stakes, treats the document as a living tool for alignment, and mentions the decision log and the partners.

Model answer. A PRD exists to align a team on what to build and why, so the best test of one is whether someone new could read it and explain the problem, the goal and what is out of scope.

Problem. Who has what problem, with evidence, and the cost of leaving it. No solution in it: "Shoppers who search by name get no results on 18% of queries, and 60% leave" and not "we need a better search."

Goals and metrics. The outcome and how we will measure it: a primary metric with a threshold, a guardrail, and a counter metric (C05).

Non-goals. Things a reasonable stakeholder might have expected that we are not doing in this version. They prevent scope creep and end arguments early.

Users and stories. Who is affected, and the key scenarios as short stories with acceptance criteria that QA can test.

Requirements. In priority order (must, should, could), each specific enough that an engineer can estimate it and a tester can test it: "results load in under 1.5 seconds at the 95th percentile," not "fast."

Risks, assumptions and open questions. The riskiest assumption and the cheap test for it, what could go wrong, and what is undecided with an owner and a date.

Launch and measurement. How it ships (staged), who decides go or no-go and on what criteria, how results will be read.

I'd scale it to the stakes: two pages for a small feature, longer with a summary on top for a multi-team launch. I'd keep a decision log at the top and update the document as we learn, and I'd review it with the engineering lead and designer before anyone commits. The mistake is writing a long document that no one reads or a feature list with no problem. Those are signals that the spec is for the author, not the team.

Follow-ups.

- **Q.** How long should a PRD be, and who should read it? **A.** "As long as the stakes require and no longer. A small change may need one page: the problem with evidence, the goal and metric, non-goals and acceptance criteria. A large cross-team launch may need ten pages plus links to research, designs and a decision log. The test is that an engineer, a designer and a stakeholder can each find what they need in a few minutes, and that someone new could explain the problem and what is out of scope. I would share it early as a draft for comment, because a PRD that was written alone and handed over arrives too late to be corrected."
- **Q.** How do you keep the PRD useful after it is written? **A.** "By treating it as a living tool. Keep a decision log at the top (date, decision, reason, who decided), update acceptance criteria when scope changes with the tradeoff stated, and link it from the tickets so that it is the place to look. Review it at the start of each sprint with engineering and design, and retire sections that no longer apply. A PRD that is not updated becomes fiction, and a team that stops reading it falls back on memory and meetings."

IQ-C16-02 · Beginner · execution · EX

Prompt. You expected a feature to take four weeks. Engineering says twelve. What do you do?

Rubric.

- **Baseline:** "Push back and ask them to find ways to be faster," or "accept it."
- **Strong:** Asks why, gets a range and assumptions, and looks at scope options.
- **Standout:** Uses the outside view, separates complexity from risk, proposes a smaller first slice and checkpoints, and avoids pressure on the estimate.

Model answer. A threefold gap between what I expected and what engineering says is information. Either my expectation was missing something about the system, or the scope is bigger than I realized. I'd start with curiosity, not pressure.

Understand. I'd ask for the breakdown: where does the time go, what is the riskiest part, what assumptions does 12 weeks rest on, and what is the range? Suppose it is 8 to 16 weeks, driven by a data model that does not exist yet. Then I'd learn that my four-week picture assumed reuse of something that is not there.

Check my side. What did I assume, and from what? If I had a reason (a similar feature took four weeks), compare with this one: what differs?

Use the outside view. How have this team's estimates compared with actuals? If their past projects ran 1.1 to 2.0 times their estimates, 12 weeks may itself be optimistic, which means planning for four is not realistic for the full scope.

Negotiate scope, not the number. If I need something in four to six weeks, ask: "What is the smallest version that delivers the core of the user's job, and how long does it take?" Perhaps a first slice that handles the common case in 4 to 5 weeks, with the full version later. Or a two-day technical spike to narrow the range before we commit.

Decide and communicate. Option 1: ship the smaller slice in five weeks and the remainder in a follow-up. Option 2: keep full scope and plan 12 to 16 weeks, with a checkpoint at week 3. I'd explain the tradeoff to stakeholders with the range and the reason. What I would not do is ask engineers to promise four weeks for the full scope. That produces hidden shortcuts and a late surprise.

Follow-ups.

- **Q.** How do you use the outside view on the estimate? **A.** "I ask how long comparable work took. If similar features in this system took on average 1.5 times the first estimate, a 12-week estimate may run to 18, and I would plan with a range, not a point. I would also ask engineering what the riskiest unknown is and whether a short spike (two to three days) could shrink the range. The outside view is not a way to pressure the estimate down; it is a way to avoid planning on the hope that this time nothing will go wrong."
- **Q.** What smaller first slice would you propose, and how would you check in? **A.** "I would ask which part delivers most of the user value with the least of the complexity, perhaps the core flow for one platform or one segment, and aim for a version in four to six weeks. Checkpoints at weeks two and four show progress against the range: if the riskiest part is on track at week two, confidence rises; if not, we re-scope early. I would agree with engineering what is cut from the first slice and when the rest would follow, and tell stakeholders the new plan with the reasons."

IQ-C16-03 · Intermediate · execution · EX · Microsoft

Prompt. You are shipping admin audit logs in an enterprise product. Legal wants 180-day retention, security wants the logs encrypted and access-controlled, finance is worried about storage cost, and support wants training. How do you coordinate?

Rubric.

- **Baseline:** "Hold a meeting with everyone and get alignment."
- **Strong:** Identifies what each partner needs and when, and sequences the work.

- **Standout:** Puts numbers on the cost tradeoff, spots a conflict between requirements (retention versus erasure requests), proposes options and a decision owner, and plans a staged rollout with design partners.

Model answer. I know the product only from public information; nothing here describes any real plan. I'd treat each partner as holding a constraint that is cheap to meet early and expensive late, and sequence accordingly.

Legal and privacy first. Retention length and the handling of personal data shape the design. They may also create a conflict: logs kept 180 days for audit may contain personal data that a person can ask to have erased in some jurisdictions [V: GDPR Article 17 right to erasure, unless retention is legally required, as summarized in guidance]. I'd ask legal for the rule, not guess, and put the answer in the spec (for example, logs record identifiers that can be pseudonymized on request, with the audit trail intact).

Security. Design review of how logs are written (tamper-resistant), who can read them (administrators only, with their own access logged), and encryption. Early involvement avoids a high-risk finding in the week before launch.

Finance. Put numbers on retention. Suppose (invented) storage costs \$4 per customer per month at 90 days and \$8 at 180 days; for 2,000 customers that is \$8,000 against \$16,000 a month, \$96,000 a year more. Options to put to finance and product leadership: include 90 days in all plans and sell 180 days as a higher tier; include 180 for all and absorb the cost; or tier by customer size. I'd bring the options with the cost and the demand evidence (how many customers need 180 days), not a request.

Support and operations. Training, scripts for the questions they will receive ("why can't I see events older than 90 days?"), and an escalation route. Involve them two to three weeks before launch.

Engineering and design. Confirm the event stream carries the data; design search and filters.

Sequence and ownership. Week 1: legal, security and finance decisions with a named owner each. Weeks 2 to 6: build. Week 5: support training. Launch: staged, first to ten design-partner customers, with go or no-go criteria in the spec (no S1 defects, search within the performance target, support trained). I'd keep a decision log and a short weekly note to all partners.

Where I'd decide. If legal and finance conflict, I'd bring the options to the shared leader with a recommendation. What would change my plan: legal saying that a particular market needs a different retention rule, which would turn retention into a regional setting.

Company note. Microsoft is the setting only: Inferred from candidate reports that enterprise-product prompts involve compliance and security partners [unverified: candidate-reported]. No Microsoft data or product behavior is claimed.

Follow-ups.

- **Q.** How do you handle the conflict between 180-day retention and erasure requests? **A.** "By asking legal for the rule and designing so that both can be met. A common approach

is to pseudonymize personal data in the logs: store tokens in place of names and emails, and keep the mapping in a separate protected store that can be deleted when a person asks to be erased, so that the audit record stays intact while the personal link is removed. Legal and privacy decide which obligation takes precedence for which data [V: GDPR Article 17 right to erasure, unless retention is legally required, as summarized in guidance], and I would record the decision and its owner. The design should not make either requirement impossible later."

- **Q.** How do you put numbers on the storage cost and stage the rollout? **A.** "Suppose 5 million events a day at about 1 KB each: 5 GB a day, 900 GB for 180 days. With three copies and indexing that might multiply the volume fivefold, about 13.5 TB, which at an assumed \$0.023 per GB-month is roughly \$310 a month. So cost is small, and the real costs are engineering time for access control and encryption and support training. I would stage the rollout with three design partners for four weeks, test retrieval and export, then release to a small share of customers before general availability, with a guardrail on query latency and access-control defects."

IQ-C16-04 · Intermediate · execution · EX

Prompt. Your engineering lead and your designer both think your plan is wrong, for different reasons. What do you do?

Rubric.

- **Baseline:** "Overrule them" or "do what they say."
- **Strong:** Listens to each, restates the shared goal, and looks for evidence.
- **Standout:** Separates the criteria each is using, finds the cheapest test, names the decision owner, commits afterward, and keeps the relationships intact.

Model answer. When two of my closest partners disagree with my plan, I'd take it as a strong signal and not as an obstacle. I'd talk to each separately first, then together.

Listen for the criteria. The engineer thinks it is too risky or too big: the criterion is cost and reliability. The designer thinks the experience will confuse users: the criterion is user value and usability. They may be seeing different parts of the same flaw. I'd ask each: "What would you do instead, and what are you worried will happen if we go with my plan?"

Restate the shared goal. "We all want [goal] by [date]. Do we agree on that?" If there is disagreement about the goal itself, that is the real conflict and I'd resolve it first.

Find the cheapest evidence. Is there a way to test the contested points quickly? A paper prototype tested with five users answers the designer's concern in two days; a technical spike answers the engineer's in two or three. I'd propose both and agree beforehand what result would change the plan.

Decide and own it. If the evidence is clear, change the plan and say what I learned. If it is not, I decide, since the call on scope and priority is mine, and I explain the reasoning and the risk I am accepting, and invite them to say what would make us revisit. Then we commit: "disagree and commit," with a date to review.

Protect the relationship. I'd thank them for the challenge, credit their input in the final plan, and make sure that raising concerns early is rewarded. What I'd avoid is deciding by rank, hiding the disagreement or reopening the decision every week. What would change my mind: any evidence that the plan fails the user or cannot be built in time.

Follow-ups.

- **Q.** What is the cheapest test for each concern? **A.** "For the designer's concern, a clickable prototype with five users, which costs two or three days and shows whether the experience confuses people. For the engineer's concern, a technical spike on the riskiest part, a day or two of building a rough version to find out how hard it is. Both are cheap compared with a quarter of building, and both turn opinions into evidence. I would agree with each of them beforehand what result would change my plan, so that the tests are real and not a formality."
- **Q.** How do you commit afterward and keep the relationships intact? **A.** "Name who decides before the test. After the evidence, the decision owner decides, I say what I learned from each of them, and everyone commits, with a record of the dissent and the signal that would reopen the decision. I would credit the engineer and designer in front of others for the changes that came from their objections. Partners who feel heard even when overruled stay willing to disagree openly next time, which is what a team needs."

IQ-C16-05 · Hard · execution · EX

Prompt. Your engineers ask for a full quarter to refactor a core system. Sales wants three features. How do you decide?

Rubric.

- **Baseline:** "Features, because they drive revenue," or "refactor, because debt is dangerous."
- **Strong:** Asks for the cost of the debt and compares it with the features.
- **Standout:** Computes payback and the effect of risk, shows sensitivity, proposes staging with a measure, and communicates the tradeoff to sales.

Model answer. I'd refuse to treat it as engineering against sales and ask the same question of both: what does each return for the capacity it uses? For the refactoring I need the principal, the interest, and the risk.

Numbers (invented). Six engineers, two-week sprints: 12 person-weeks a sprint. The refactor costs 24 person-weeks (two sprints, the principal). Workarounds cost about 10% of capacity, 1.2 person-weeks per sprint (the interest). Payback on capacity alone: $24 \div 1.2 = 20$ sprints, about 40 weeks. Over a year (26 sprints), capacity savings are $26 \times 1.2 = 31.2$ person-weeks, more than the 24 spent: a net of +7.2 person-weeks on time savings alone, and payback takes the first 20 sprints of the year.

But debt also carries risk. Suppose the system has two serious incidents a year, each costing \$150,000 in lost revenue and recovery, and the refactor would roughly halve them: \$150,000 a year of avoided loss. At about \$15,000 per person-week of engineering value (invented), the time savings are $31.2 \times \$15,000 = \$468,000$, plus \$150,000 of avoided incidents,

\$618,000 a year, against a cost of $24 \times \$15,000 = \$360,000$. So the year-one return is about +\$258,000, and over two years it is \$1,236,000 against \$360,000, a gain of \$876,000. The case is positive in year one and strong over two, if the interest and the incident rate hold.

Compare with the features. Size the three sales features the same way (C15.5): expected value per person-week. If they return more than \$15,000 per person-week, they win this year; if less, the refactor is at least as good.

Sensitivity. The interest and the incident rate are the uncertain numbers. If interest is 5% rather than 10%, the time savings halve to \$234,000 and the year-one return is about break-even (+\$24,000), and if the incident saving also disappears it is −\$126,000. So I'd ask the team to measure the interest: hours lost to the system over the past quarter, and the incident history.

Decision. Not a whole quarter of nothing else. Stage it: do the highest-interest part first, perhaps 8 person-weeks, with a measure of hours saved and incidents after, and ship the most valuable one or two sales features in the same quarter. Tell sales what they get and when, and why the refactor is part of reliability they rely on. Review at the end of the first stage. What would change my mind: a major incident in the system, which would move the whole refactor up, or a sales feature tied to a signed commitment.

Follow-ups.

- **Q.** The refactor pays back in about 20 sprints. How do you decide? **A.** "On the full picture, not payback alone. The principal is 24 person-weeks and the interest is about 1.2 person-weeks per sprint of workarounds, so payback on workaround time alone is about 20 sprints, around nine months, which is long. Then add risk: if the old system has a 5% chance per quarter of an outage that costs \$400,000, the expected loss is about \$20,000 a quarter. I would propose a staged slice: refactor the module that causes most of the workarounds in six weeks, measure the drop in workaround time and incidents, and decide on the rest with evidence."
- **Q.** How do you communicate the tradeoff to sales? **A.** "Directly and with dates. I would say: this quarter we deliver two of the three features and a six-week slice of the refactor, because the system's instability puts the revenue at risk and slows every later feature; the third feature follows next quarter. I would show the revenue each feature is tied to and what the refactor protects, and ask which feature matters most to open deals. Sales needs a clear answer and a date they can give customers, and a promise I cannot keep is worse than a later date."

IQ-C16-06 · Hard · execution · EX

Prompt. A teammate suggests using an AI tool to write the PRD and summarize the user research for you. What do you think, and what is your approach?

Rubric.

- **Baseline:** "Great, it saves time," or "I wouldn't use it."
- **Strong:** Names what it is good for and what it is not, with verification.

- **Standout:** Distinguishes drafts from evidence, protects confidential data, keeps the decision and the research with people, labels the work, and gives a concrete workflow.

Model answer. [as of 2026-10] I'd use it for speed on drafts and structure, and I'd be strict about what counts as evidence. A model's output is a draft or a hypothesis. It is not a source.

For the PRD. I'd write the problem, the goal and the non-goals myself, because they are my judgment, and give the tool my bullet points to turn into a first draft, to suggest missing sections, to list edge cases and to argue against the plan (a red team). Then I'd edit hard: check every number, every claim about users and every technical statement against a source, and remove generic text that says nothing specific.

For the research. The evidence is the participants and the transcripts (C06). I could use a tool to organize notes I give it, to propose themes and to find quotes, but I'd check each theme against the transcripts and quote only what a person actually said. A summary is a pointer to the transcripts, not a replacement for them. And a model-generated "user" is not a user: its answers say what text on the internet looks like, not what my customers do.

Data and privacy. I'd follow the company's policy: no customer data, unreleased plans or personal information in tools that are not approved for them. For approved tools, I'd still strip identifiers when I can.

Ownership and honesty. The decision, and the accountability, stay with me and the team. I'd say in the document where AI helped and not present synthesized text as findings. If the PRD contains a claim I cannot trace to a source or my own data, it comes out.

What I'd tell my teammate. "Great for drafting and for finding gaps. For anything we rely on, we verify against a source, and the research stays with real users." What would change my practice: a tool approved for confidential data and with traceable citations to the underlying sources, in which case I'd still verify the citations.

Follow-ups.

- **Q.** What is your concrete workflow for using an AI tool on a PRD? **A.** "I write the problem, goal and non-goals myself. Then I give the tool my notes to produce a first draft, a list of edge cases and missing sections, and a plain-language rewrite. I check each factual claim against its source, mark AI-drafted sections in the document, and have an engineer and a designer review it, since they will find errors I would not. I do not paste confidential data into a public tool; I use only a tool my company approves for this data. The decision and the evidence stay with people."
- **Q.** How do you check an AI summary of user research? **A.** "I read the raw notes or transcripts for at least a sample and test the summary against them. I pick five key claims in the summary, find the supporting quote for each, and look for what is missing, such as a minority view or a contradiction. If more than one of five claims has no support, I discard the summary and summarize it myself. I keep the raw material and quotes in the PRD's appendix, so that others can check my reading. A summary is a starting point for my own judgment, not a replacement for reading."

IQ-C16-07 · Hard · execution · EX

Prompt. It is the day before launch. QA finds a bug that loses user data for 0.5% of users, with a three-day fix. Leadership wants to ship on time. What do you do?

Rubric.

- **Baseline:** "Ship and fix it later," or "delay."
- **Strong:** Classifies the severity, sizes the harm and weighs the delay.
- **Standout:** Applies a pre-set release rule, considers staging and its limits (staging still exposes some users), identifies alternatives that remove the harm without delay, and communicates clearly with a go or no-go owner.

Model answer. Data loss is an S1 under any sensible release rule: it harms users irreversibly. My starting position is no-go for a full launch until it is fixed or contained, and I'd check that the rule agreed in advance says the same ("no open S1").

Size the harm. With 200,000 users in the first month (invented), 0.5% is 1,000 people losing data. Even a staged release does not make that safe: at 5% exposure, 50 people could lose data, which I would not accept as a planned outcome. The harm is not recoverable by a patch after the fact.

Size the delay. A three-day delay costs something real: say \$15,000 a day of expected revenue and a commitment to partners (invented), \$45,000 and some reputational cost. That is small beside 1,000 people's data and the trust cost.

Look for ways to contain without delaying. (1) Can the bug be avoided by disabling the feature path that triggers it, via a feature flag, and launching the rest? (2) Can we launch to users who do not hit the affected condition, with a safe fallback? (3) Can the fix be done in less than three days, or the launch moved by one day with partial fixes verified? (4) Can we warn and offer a backup before the risky action? Any option that removes the data-loss path is acceptable; one that merely reduces it is not.

Decision and communication. Recommend: hold the launch for the fix, or launch with the risky path disabled if that is feasible and verified. Tell leadership the facts in five lines: what the bug does, how many users it could hit, the options with cost and date, my recommendation, and the go or no-go owner (me with the engineering lead, with the sponsor informed). Tell partners and support the new date or the reduced scope.

Afterward. Add a regression test for the case, and look at why testing found it late. What would change my mind: evidence that the affected condition is much rarer than assumed or that the data can be recovered automatically.

Follow-ups.

- **Q.** What alternatives could remove the harm without delaying the launch? **A.** "Several, and I would ask engineering for each: turn off the specific feature path that triggers the bug using a feature flag and ship the rest; block the triggering action with a warning until the fix lands; add a client-side backup before the risky operation; or launch only to the group that cannot hit the path. Each must be tested to be safe, because a workaround

that leaves a gap is not a solution. If one works, we can keep the date and ship the fix three days later. If none works, the launch moves."

- **Q.** Who decides, and how do you communicate? **A.** "The release owner under the rule agreed in advance, which says no open severity-one bug. I bring facts: 0.5% of 200,000 first-month users is about 1,000 people losing data, a staged 5% release would still expose about 50, the fix takes three days, and the options above. I recommend no-go for a full launch unless a containment option is verified. I put the recommendation and the decision in writing, tell support and customers' contacts what is delayed and why, and record the rule that applied, so that the next team has it."

12. Exercises

Ten exercises. Each has a model solution and an alternative. Figures are assumptions unless marked, and the arithmetic was computed.

EX-C16-01 · Rewrite the problem statement

Problem. Rewrite each as a problem statement with no solution in it, using who, what happens and the cost: (a) "We need a better onboarding." (b) "Add a dark mode." (c) "Users want notifications." (d) "Make checkout faster."

Context. Four requests have arrived in a ticket queue.

Data. Invent plausible figures and mark them (invented).

Constraints. Six minutes.

Expected approach. The good-versus-bad test (FW-C16-02), test 1.

Solution. (a) "New users who sign up on a phone abandon the app within the first session 55% of the time (invented); the biggest drop is at the permission prompts and the empty home screen." (b) "People who use the app in the evening report eye strain and 12% of low-rated reviews mention brightness (invented)." (c) "Users who miss a time-sensitive event (a delivery, a price change) cannot learn of it without opening the app; 30% of support contacts ask for status (invented)." (d) "At checkout, 22% of people who start do not finish, and recordings show most stall on the address form (invented)."

Alternative solution. Keep the original line and add the problem under it, so the requester can see their request is understood.

Tradeoffs. Problem statements invite other solutions, which can unsettle a requester who had one in mind; explain why.

EX-C16-02 · Write the non-goals

Problem. For a feature that lets users save search filters, write three non-goals that a reasonable stakeholder might have expected in scope.

Context. A first release.

Data. None.

Constraints. Four minutes.

Expected approach. C16.2, non-goals.

Solution. (1) Notifications when new results match a saved filter (a later release if usage justifies). (2) Sharing saved filters with other people. (3) Syncing saved filters across devices in version 1 (stored on the device only). Each is plausible for a stakeholder to expect; stating them avoids scope creep, and the third also shortens the build.

Alternative solution. Add a fourth, "no editing of a saved filter's name after creation," if it saves effort and costs little to users.

Tradeoffs. Too many non-goals can make the feature feel thin; choose those whose absence the first users will accept.

EX-C16-03 · Split a story and write acceptance criteria

Problem. Split "As a user, I want to manage my notifications" into three stories using different splitting patterns, and write given-when-then criteria for one.

Context. The story is too big to estimate.

Data. None.

Constraints. Seven minutes.

Expected approach. FW-C16-03, INVEST.

Solution. By workflow step: (1) turn all notifications off, (2) choose which categories to receive, (3) set quiet hours. Or by user type: (1) a person with one device, (2) a person with several devices. Criteria for (3): *Given* I have notifications on, *when* I set quiet hours from 22:00 to 07:00, *then* no notification sounds or vibrates in that period and they appear in the notification list afterward. *Given* a time-critical alert marked as urgent, *when* it arrives in quiet hours, *then* it still sounds (if I have allowed urgent alerts). *Given* I change time zones, *when* quiet hours apply, *then* they follow my local time.

Alternative solution. Split by simple-then-complex: a single global switch first, then categories, then schedules.

Tradeoffs. Splitting by technical layer ("database," "interface") yields stories no user can test.

EX-C16-04 · Plan with the outside view

Problem. Past projects of a team ran 1.2, 1.3, 1.5, 1.8, 1.1, 2.2 and 1.4 times their estimates. A new project is estimated at 6 weeks. Compute the median and mean ratios, a planning range, and say what you would do if the date is 7 weeks.

Context. The sponsor wants 7 weeks.

Data. As above.

Constraints. Five minutes.

Expected approach. FW-C16-04.

Solution. Sorted: 1.1, 1.2, 1.3, 1.4, 1.5, 1.8, 2.2; the median is 1.4 and the mean is $10.5 \div 7 = 1.5$. At 6 weeks: median plan 8.4 weeks, mean plan 9 weeks, and the worst past case (2.2) 13.2 weeks. A date of 7 weeks sits below the median. Options: cut scope so that the estimated work is about 4.5 weeks ($4.5 \times 1.5 = 6.8$), run a spike to narrow the range, and set a checkpoint at week 3.

Alternative solution. Commit to 7 weeks only for the core scope and name what slips if the checkpoint shows an overrun.

Tradeoffs. Using the history is unpopular because it is longer; it is also more often right.

EX-C16-05 · Technical-debt payback

Problem. A team of 5 engineers works in 2-week sprints (10 person-weeks per sprint). A fragile module costs 12% of capacity in workarounds. Fixing it costs 9 person-weeks. Compute the interest, the payback, the net over 13 sprints and the break-even interest.

Context. The engineering lead asks for the work.

Data. As above.

Constraints. Five minutes.

Expected approach. FW-C16-05.

Solution. Interest: $12\% \times 10 = 1.2$ person-weeks per sprint. Payback: $9 \div 1.2 = 7.5$ sprints. Net over 13 sprints: $13 \times 1.2 - 9 = 15.6 - 9 = +6.6$ person-weeks. Break-even interest: $9 \div 13 \div 10 = 6.9\%$ of capacity. If the true interest is 5%, the module costs 0.5 per sprint, the net is $6.5 - 9 = -2.5$, and the fix does not pay within the 13 sprints (six months). So the decision depends on measuring the interest.

Alternative solution. Stage the fix: do the highest-cost third first (3 person-weeks) and measure the interest saved before continuing.

Tradeoffs. Fixing all debt is as wrong as fixing none; fix where the interest is highest.

EX-C16-06 · Triage for release

Problem. Ten days before launch there are 2 S1 defects, 7 S2 (4 on the core path) and 20 S3. Two engineers have 12 person-days. S1 fixes take about 2 days each; core-path S2 fixes about 1.5 days each. The rule: no open S1; S2 only with a workaround and an owner. Compute whether the plan fits and what happens if S1 fixes take 3 days.

Context. A go or no-go decision.

Data. As above.

Constraints. Six minutes.

Expected approach. C16.9 and Worked example 5.

Solution. S1: $2 \times 2 = 4$ person-days. Core-path S2: $4 \times 1.5 = 6$. Total 10 of 12, leaving 2 person-days (17%). The 3 non-core S2 ship with workarounds, and the 20 S3 go to the backlog. If S1 fixes take 3 days: $2 \times 3 + 6 = 12$, no buffer at all; any slip breaks the plan. Then: re-estimate after the first S1 fix, and be ready to launch to a small group first, drop the least severe core-path S2 with a workaround, or take two more days.

Alternative solution. Ask whether a feature flag can disable the paths with the S2 defects and keep the launch date.

Tradeoffs. A buffer of 17% is thin; a stage with a guardrail protects against what the plan missed.

EX-C16-07 · Map the partners

Problem. A new "buy now, pay later" option at checkout is planned. List the partners to involve, what each needs, and when.

Context. A six-month project.

Data. None.

Constraints. Seven minutes.

Expected approach. The partner table in C16.7.

Solution. Legal and compliance (credit regulation, disclosures, terms; before design is final); privacy and security (credit data, fraud; at design review); finance (loss rates, fees, funding; at the start, since economics decide viability); risk and fraud teams (underwriting rules, limits; early); support (what customers will ask about payments and disputes; weeks before launch); marketing (how it is described, with disclosures; when the plan forms); design and accessibility (clear terms and totals; during design); data science (default rate measurement and experiment plan; before the build); engineering and the payment partner (integration). Late legal or finance involvement is the most expensive; both can end the project.

Alternative solution. Add a pilot with a small group and a licensed partner to learn before the full build.

Tradeoffs. Involving everyone early slows the start; skipping any of legal, finance or risk can stop the launch.

EX-C16-08 · Respond to "not feasible"

Problem. An engineer says, "That can't be done in this release." Write a response that gets information, and three questions to ask.

Context. A planning meeting.

Data. None.

Constraints. Four minutes.

Expected approach. C16.4, handling pushback.

Solution. Response: "Thanks for flagging it. Help me understand what makes it hard, so I can adjust the plan." Questions: (1) What is the hardest part, and is there a cheaper way to achieve the same user outcome? (2) What would make it feasible: more time, a smaller scope, a dependency resolved, a technical spike? (3) What would you cut if we had half the time? Then restate the goal and offer scope options. Avoid "just try," "it's only a small change," or appeals to the date.

Alternative solution. Ask the engineer to propose the version they would be proud to ship in the time available.

Tradeoffs. Listening takes longer than pushing, and it surfaces constraints you would otherwise meet in week six.

EX-C16-09 · AI tools: yes, with conditions, or no

Problem. For each task decide whether to use an AI tool, and with what conditions: (a) draft a spec from your bullet points; (b) list edge cases for a feature; (c) summarize ten customer interviews into themes; (d) generate five "user personas" and treat them as your target users; (e) summarize a spreadsheet of customer complaints that contains names and email addresses.

Context. [as of 2026-10]

Data. None.

Constraints. Six minutes.

Expected approach. FW-C16-08.

Solution. (a) Yes: edit and verify every claim. (b) Yes: a good use; check which cases matter. (c) With conditions: use it to propose themes from transcripts you supply, check each theme against the transcripts, and quote only real statements; the interviews remain the evidence. (d) No as evidence: invented personas are not users; use them, if at all, as prompts for questions to take to real people. (e) Only with an approved tool and with identifiers removed, per company policy.

Alternative solution. For (c), have a teammate independently read three transcripts and compare with the summary.

Tradeoffs. The tools save time on drafts and cost trust if unverified claims slip through.

EX-C16-10 · A timed full answer

Problem. Give a 20-minute answer to: "Walk me through how you would take a new 'schedule a send' feature for an email product from idea to launch."

Context. Self-scored; record yourself.

Data. None. Label what you assume.

Constraints. Use the 20-minute budget in section 4.

Expected approach. The execution structure, scored against the six behaviors in section 1.

Solution. An outline of a strong answer. Clarify the goal: fewer late-night emails sent, or more replies at good times. Spec: problem (people write emails when it suits them and the recipient reads them when it suits the recipient), metric (share of sends that are scheduled; reply rate for scheduled sends; guardrail: send failures), non-goals (recurring sends, scheduling from shared inboxes), requirements in priority order, riskiest assumption (people will use it; check logs for late-night sends). Partners: engineering for the scheduler and failure handling, design for the picker and the states, data science for the measurement, privacy for stored drafts, support for "my email did not send." Estimates as ranges with a spike for the scheduler. Quality bar: an email must never be lost or sent twice (S1), test across time zones and daylight changes. Staged launch with a flag, a rollback and go or no-go criteria.

Alternative solution. Start from the press release the user would read and work backwards to requirements.

Tradeoffs. A longer spec adds precision and loses readers; keep two pages and a decision log.

13. Practice protocol

Timed drills (three a week for three weeks).

- *Week 1: the spec.* Use IQ-C16-01, EX-C16-01, EX-C16-02 and EX-C16-03. Write a two-page spec for a feature you use, with the problem, a metric, non-goals and stories, then run the eight tests in FW-C16-02.
- *Week 2: engineering and design partnership.* Use IQ-C16-02, IQ-C16-04, IQ-C16-05, EX-C16-04, EX-C16-05 and EX-C16-08. Do the estimate and debt arithmetic by hand and say the conversation aloud.
- *Week 3: partners, quality and tools.* Use IQ-C16-03, IQ-C16-06, IQ-C16-07, EX-C16-06, EX-C16-07, EX-C16-09 and EX-C16-10. Make a go or no-go call with numbers, and run a full 20-minute answer with a partner who raises a conflict midway.

With a partner. Ask your partner to play the engineering lead who says "that's a quarter" and the designer who says "that will confuse people." Practice asking for the range, the riskiest part and the criteria, and not defending.

Review a real spec. Read a spec from your team or a public design document, and run the eight tests on it. Write down what you would change.

Feedback checklist (yes, partly, no).

1. Did my spec separate the problem from the solution and state a metric with a guardrail?
2. Did I name non-goals a stakeholder might have expected?
3. Were requirements testable and in priority order?
4. Did I treat estimates as ranges, check the outside view, and negotiate scope?
5. Did I involve design, data science and the other partners at the right time?

6. Did I handle disagreement with the shared goal, criteria and cheap evidence?
7. Did I discuss technical debt with principal, interest and payback?
8. Did I set a quality bar, severity rules and a go or no-go owner?
9. Did I describe AI tool use with verification and data protection?
10. Did I finish with the two-sentence close from section 4?

Answer journal. For each drill record the date, the prompt, the spec sections you covered, the friction you handled, your score out of ten and the step you rushed.

14. Implementation playbook

Apply the chapter on real work over the next three weeks. Each deliverable doubles as an interview story.

Days 1 to 3. Take the last spec your team wrote (or a ticket for an upcoming piece of work) and run the eight tests in FW-C16-02. Rewrite the problem statement and add two non-goals. Share the rewrite with the engineering lead and designer and note what changes.

Week 1. For the next piece of work, write a two-page spec with the skeleton (FW-C16-01) and a decision log. Review it with engineering and design before anyone commits, and keep a list of what they changed.

Week 1. Ask engineering for a range and the riskiest part on an upcoming estimate. Compare their past estimates with actuals for the last five projects and compute the ratio (the outside view).

Week 2. Choose one area of technical debt and run the conversation: measure the interest (hours or incidents), estimate the principal, and compute payback with a sensitivity. Present it next to a feature of similar size.

Week 2. Draw the partner map for an upcoming launch (C16.7) and book the first conversation with each partner who is not yet involved.

Week 3. Set a release rule for the next launch: severity definitions, what must be true to ship, who makes the call. Write it in the definition of done and share it before testing begins.

Deliverables that become stories: the rewritten spec (a time you made a team's work clearer), the estimate and debt analyses (a time you worked with engineers using evidence), and the release rule (a time you set a bar before the pressure came). Keep the numbers real, and keep what you did separate from what the team did.

15. Key takeaways and further reading

Key takeaways

1. A good spec separates the problem from the solution, states a metric with a guardrail and explicit non-goals, and lists testable requirements in priority order; its job is alignment, and its length matches the stakes.
2. Treat engineering estimates as ranges with assumptions, check them against history, and negotiate scope, not the number; adding people to a short project rarely helps.
3. Discuss technical debt as an investment with principal, interest, payback and a sensitivity, compared with feature work, and stage the repayment.
4. Settle disagreements among product, design and engineering by restating the shared goal, naming each side's criteria, finding the cheapest evidence and deciding with clear ownership.
5. Set the quality bar and severity rules before the pressure, involve legal, security, support and other partners at the right time, and make the release call with the facts; use AI tools for speed with verification, data protection and human ownership of the decision.

Further reading

- Marty Cagan, *Inspired* (2nd edition, 2017) and *Empowered* (2020). On how strong product teams work with engineering and design.
- Mike Cohn, *User Stories Applied* (2004), and Bill Wake's INVEST checklist [V: Cohn: Addison-Wesley, March 2004, ISBN 9780321205681; Wake: INVEST in Good Stories, and SMART Tasks, XP123, 2003]. On stories and acceptance criteria.
- Frederick P. Brooks, Jr., *The Mythical Man-Month* (anniversary edition, 1995). On estimates, staffing and communication [V: Brooks, 1975 with editions 1982 and 1995].
- Colin Bryar and Bill Carr, *Working Backwards* (2021). Amazon's practice of writing the press release and FAQ first [V: St. Martin's Press, 9 February 2021, ISBN 9781250267597; includes a chapter on managing inputs, not outputs].
- Betsy Beyer and others (eds.), *Site Reliability Engineering* (2016). Reliability, release practice and postmortems [V: Site Reliability Engineering, O'Reilly, 2016].
- Kent Beck with Cynthia Andres, *Extreme Programming Explained* (2nd edition, 2004). The origins of many agile practices [V: 2nd edition, Addison-Wesley, 16 November 2004, ISBN 9780321278654 (with Cynthia Andres); first edition 1999].
- US Government Accountability Office, report GAO-14-694 (31 July 2014), testimony GAO-14-824T (31 July 2014), and the security report GAO-14-730 (16 September 2014), all at gao.gov. The primary sources for CS-C16-02.
- Your company's policy on approved AI tools and data handling [as of 2026-10].